

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-a8fd78388395474ac.jsonl`  
- SHA-256: `8af5cdad951700e5b30b771be4d977ca759ffce8657165f2d84620dbde68092`  
- Source modified: `2026-06-13T19:15:44+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_095035a0-e9f`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T19:14:42.411Z)

Adversarially VERIFY this fusion-P2 review finding by reading the actual code. Try to REFUTE it; only confirm real=true if it genuinely holds against the code. Default to real=false if uncertain or if it's a nitpick/false-positive.

FINDING: {"title":"Gate-A net-crossing axis/net is re-estimated over the whole trajectory once per candidate window","severity":"low","file":"backend/pipeline/segmenters/fusion_hybrid.py:77","detail":"`\_enrich\_candidate\_stats` calls `rally\_gate.gate\_windows(points, [(cw['start'], cw['end'])]), fps, min\_crossings=0)` separately for every window. `gate\_windows` internally calls `estimate\_play\_axis(points)` (sorts all visible xs and ys) and `\_points\_in(points, ...)` (full scan) on the ENTIRE whole-video trajectory each time ? so net/axis estimation and a full point scan are repeated O(num_windows) times. For a long match with many candidates and a dense trajectory this is wasteful (O(windows x points x log points)). Correctness is fine (net/axis is documented as stable over the whole trajectory, so recomputing yields the same value); it is purely a performance smell on real footage, which runs owner-side on GPU/WSL.", "fix":"Estimate axis/net once (e.g. cache `estimate\_play\_axis(points)` / a precomputed deadband on the instance for the duration of process_video) and pass all window intervals to a single `gate\_windows(points, intervals, ...)` call, indexing the result per window ? instead of one call per window."}

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:14:45.113Z)

I'll verify this finding by reading the actual code. Let me start by examining the relevant files.

3. user (2026-06-13T19:14:46.067Z)

```
1      """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).  
2  
3      Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,  
4      TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`  
5      do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
```

```

6      shuttle-seeded window via two overridable seams the engine exposes:
7
8      - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9        GPU-free, computed from the trajectory we already have); and, only when the person cue
10       is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11       (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12       fusion (P3). The person path lazily builds ONE ``PersonDetector`` ? so the default path
13       never imports torch / touches the GPU.
14       - ``_select_for_handoff`` ? optional served Gate A/B: when
15       ``indexing.fusion.min_crossings``
16       (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
17       are
18       dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19       **Persisted candidates remain the full superset** regardless, so the offline
20       experiment
21       harness can re-score any variant.
22
23       Defaults reproduce the substrate exactly: net_crossings/person features are persisted
24       but
25       nothing is gated (thresholds 0, person OFF), so ``FusionSegmenter`` with default config
26       seeds
27       and hands off identically to ``wasb_hybrid``. Registered as ``fusion_hybrid``, NOT the
28       default
29       segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
30       through the
31       experiment harness (EXPERIMENT_HARNESS.md), never silently.
32       """
33       from typing import Any, Dict, List, Optional, Tuple
34
35       from backend.pipeline.segmenters.base import segmenter_registry
36       from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
37       from backend.pipeline.detectors.base import DetectorRunner
38       from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
39       from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
40       from backend.pipeline.detectors import rally_gate
41       from backend.pipeline.detectors import fusion_features as ff
42
43       class FusionSegmenter(TrajectoryHybridSegmenter):
44           """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
45           B)."""
46
47           family = "fusion"
48           display_label = "Fusion"
49
50           def __init__(self, db: Any, config: Any):
51               super().__init__(db, config)
52               # Built lazily only if the person cue is enabled (keeps torch/GPU off the
53               default path).
54               self._person: Optional[Any] = None
55
56           @property
57           def name(self) -> str:
58               return "fusion_hybrid"
59
60           @property
61           def description(self) -> str:
62               return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
63               rallies, "
64
65                       "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
66               and the "
67
68                       "AI provider sets semantic boundaries.")
69
70           def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
71               Any]]:

```

```

59         substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
60         if substrate == "tracknet":
61             cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
62             return TrackNetRunner(cfg), {
63                 "weights_path": cfg.weights_path,
64                 "queue_length": cfg.queue_length,
65                 "fusion_substrate": "tracknet",
66             }
67         wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
68         return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
"wasb"}
69
70     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
71                                frame_width: float, video_path: str,
72                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
73         stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
video_path, idx_cfg)
74         # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
persisted
75         # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
estimated
76         # over the whole trajectory by rally_gate (camera-agnostic).
77         gated = rally_gate.gate_windows(points, [[cw["start"], cw["end"]]], fps,
min_crossings=0)
78         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
79
80         fcfg = idx_cfg.get("fusion", {})
81         if fcfg.get("cue_flags", {}).get("person", False):
82             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
83         return stats
84
85     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
86                          frame_width: float, video_path: str,
87                          fcfg: Dict[str, Any]) -> Dict[str, float]:
88         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
89         frame range and compute the scale/translation-invariant person features. Lazily
builds
90         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
91         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
92         from backend.pipeline.detectors.person_detector import PersonDetector
93         if self._person is None:
94             self._person = PersonDetector(self.config)
95
96         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
97         start_f, end_f = int(cw["start"] * fps), int(cw["end"] * fps)
98         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
99         fw = det["frame_width"] or frame_width
100        fh = det["frame_height"] or 0.0
101
102        poly = fcfg.get("court_polygon")
103        polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
104        net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
105        shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
106        return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
107                                         court_polygon=polygon, net=net)
108
109     def _select_for_handoff(self, windows: List[Dict[str, Any]],
110                            window_stats: List[Dict[str, Any]],
111                            idx_cfg: Dict[str, Any]) -> List[int]:
112         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds

```

```

113         (0) keep every window ? identical to the substrate. Persisted candidates are
114         untouched (full superset)."""
115         fcfg = idx_cfg.get("fusion", {})
116         min_crossings = int(fcfg.get("min_crossings", 0))
117         min_players = int(fcfg.get("min_players", 0))
118         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120         keep: List[int] = []
121         for i, st in enumerate(window_stats):
122             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                 continue
124             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                 continue
126             keep.append(i)
127         return keep
128
129
130     segmenter_registry.register(FusionSegmenter)
131

```

4. user (2026-06-13T19:14:46.745Z)

warning: in the working copy of 'backend/pipeline/segmenters/trajectory_hybrid.py', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'tests/test_trajectory_hybrid_equivalence.py', LF will be replaced by CRLF the next time Git touches it

```

backend/config/models.py | 34 ++++++
backend/pipeline/detectors/person_detector.py | 47 ++++++
backend/pipeline/segmenters/__init__.py | 2 +
backend/pipeline/segmenters/trajectory_hybrid.py | 50 ++++++-----
docs/README.md | 3 +-
tests/test_config.py | 30 ++++++
tests/test_trajectory_hybrid_equivalence.py | 22 ++++++
7 files changed, 177 insertions(+), 11 deletions(-)

```

5. assistant (2026-06-13T19:14:50.603Z)

Now let me read the `rally\_gate` module to verify the claim about `gate\_windows`, `estimate\_play\_axis`, and `\_points\_in`.

6. user (2026-06-13T19:14:51.174Z)

```

1     """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3     WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4     the opponent for service after losing a point, that single trajectory looks like a
5     rally. The discriminator is structural, not duration-based: a real rally has the
6     shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7     single service feed crosses ~once.
8
9     This module is camera-agnostic and needs NO new model ? it works on the trajectory
10    we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11    axis with the larger shuttle spread) and the net position (median shuttle coord on
12    that axis ? the shuttle clusters at the two ends and crosses the net region often,
13    so the median sits near the net), then counts sign changes of (coord - net) across
14    the visible points inside a candidate window, with a deadband to ignore jitter.
15
16    Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17    Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18    windows from trajectory_to_action_windows, or to fold into it later.
19    """
20    from __future__ import annotations
21

```

```

22     from dataclasses import dataclass
23     from typing import List, Sequence, Tuple
24
25     Interval = Tuple[float, float]
26
27
28     @dataclass
29     class GatedWindow:
30         start: float
31         end: float
32         crossings: int
33         visible_points: int
34         kept: bool
35
36
37     def _spread(vals: Sequence[float]) -> float:
38         """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39         if len(vals) < 2:
40             return 0.0
41         s = sorted(vals)
42         lo = s[max(0, int(0.05 * (len(s) - 1)))]
43         hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44         return hi - lo
45
46
47     def _median(vals: Sequence[float]) -> float:
48         if not vals:
49             return 0.0
50         s = sorted(vals)
51         n = len(s)
52         return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55     def estimate_play_axis(points) -> Tuple[str, float, float]:
56         """Return (axis 'x'|'y', net_value, span) from visible points.
57
58         Play axis = the image axis along which the shuttle travels most (larger spread).
59         For a baseline camera that's Y (players top/bottom); for a side camera, X.
60         """
61         xs = [p.x for p in points if p.visible]
62         ys = [p.y for p in points if p.visible]
63         sx, sy = _spread(xs), _spread(ys)
64         if sx >= sy:
65             return "x", _median(xs), sx
66         return "y", _median(ys), sy
67
68
69     def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70         """Count sign changes of (coord - net) across visible points, ignoring the
71         deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72         crossings = 0
73         last_side = None
74         for p in window_points:
75             if not p.visible:
76                 continue
77             v = (p.x if axis == "x" else p.y) - net
78             if abs(v) < deadband:
79                 continue
80             side = v > 0
81             if last_side is not None and side != last_side:
82                 crossings += 1
83             last_side = side
84         return crossings
85
86

```

```

87     def _points_in(points, fps: float, start: float, end: float):
88         return [p for p in points if start * fps <= p.frame <= end * fps]
89
90
91     def gate_windows(points, windows: List[Interval], fps: float,
92                     min_crossings: int = 2, deadband_frac: float = 0.06) ->
List[GatedWindow]:
93         """Annotate each (start,end) window with its shuttle net-crossing count and a
94         keep decision (crossings >= min_crossings). Deadband is a fraction of the play
95         axis span. Net/axis are estimated once over the whole trajectory (stable)."""
96         axis, net, span = estimate_play_axis(points)
97         deadband = deadband_frac * span
98         out: List[GatedWindow] = []
99         for (s, e) in windows:
100             wp = _points_in(points, fps, s, e)
101             c = count_net_crossings(wp, axis, net, deadband)
102             vis = sum(1 for p in wp if p.visible)
103             out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
104                                   kept=c >= min_crossings))
105         return out
106
107
108     def apply_gate(points, windows: List[Interval], fps: float,
109                   min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
110         """Convenience: return only the windows that pass the crossing gate."""
111         return [(g.start, g.end) for g in
112                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
113

```

7. assistant (2026-06-13T19:14:55.178Z)

Now I've confirmed the key facts about `gate\_windows`. Let me verify the calling pattern: that `\_enrich\_candidate\_stats` is called once per window. Let me look at `trajectory\_hybrid.py` to see how the enrich seam is invoked.

8. user (2026-06-13T19:14:55.782Z)

```

1     """Shared engine for trajectory-substrate hybrid segmenters.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4     standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5     now the single home for the whole pipeline shape:
6
7         healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8         -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10    A concrete subclass supplies only its identity and runner wiring:
11    - ``family`` ? config section under ``indexing.*``, output subdir, and
12      the ``<family>-hybrid-<model>`` detector tag in metadata.
13    - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14    - ``name`` / ``description`` properties (the registry contract).
15    - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16      detection_params extras such as weights_path).
17
18    This base is deliberately NOT registered; only concrete subclasses register.
19    """
20    import os
21    import time
22    import logging
23    import concurrent.futures
24    from typing import Any, Dict, List, Optional, Tuple
25
26    import cv2
27
28    from backend.pipeline.segmenters.base import BasePlaySegmenter

```

```

29     from backend.utils.video import get_video_duration, extract_video_chunk
30     from backend.sports import sport_registry
31     from backend.providers import provider_registry
32     from backend.pipeline.detectors.base import DetectorRunner
33
34     logger = logging.getLogger(__name__)
35
36
37     def _fmt_ts(seconds: float) -> str:
38         seconds = max(0, int(seconds))
39         h, rem = divmod(seconds, 3600)
40         m, s = divmod(rem, 60)
41         return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44     class TrajectoryHybridSegmenter(BasePlaySegmenter):
45         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46 boundaries."""
47
48         #: config section under `indexing` (velocity_threshold lives there), the
49         #: output subdir for trajectories, and the metadata detector-tag prefix.
50         family: str = ""
51         #: human-readable label used in log lines.
52         display_label: str = ""
53
54         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55 Any]]:
56             """Return (runner, detector-specific detection_params extras)."""
57             raise NotImplementedError
58
59         @staticmethod
60         def _probe_fps_width(video_path: str) -> tuple:
61             """Return (fps, frame_width) for a video, with sensible fallbacks."""
62             cap = cv2.VideoCapture(video_path)
63             fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
64             width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
65             cap.release()
66             return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
67
68         @staticmethod
69         def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
70             """Provider-reported exchange count, else a duration-based estimate.
71
72             One canonical implementation ? the twins had drifted (wasb relied on
73             ``int(None)`` raising into the fallback; same result, by accident).
74             """
75             shot_count = segment.get("shuttle_exchange_count")
76             if shot_count is None:
77                 return max(3, int(duration / 0.8))
78             try:
79                 return int(shot_count)
80             except (ValueError, TypeError):
81                 return max(3, int(duration / 0.8))
82
83         # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
84         -----
85         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
86 frame_width: float, video_path: str,
87 idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
88             """Stats dict persisted into ``candidate_segments.stats`` for one window. The
89 BASE
90 returns exactly the 4 motion keys, so the trajectory twins persist byte-
91 identical
92 rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
93 features). ``points`` are the whole-video trajectory points

```

```

(TrajectoryPoint)."""
89         return {
90             "peak_velocity": cw["peak_velocity"],
91             "mean_velocity": cw["mean_velocity"],
92             "active_frame_count": cw["active_frame_count"],
93             "track_id_count": cw["track_id_count"],
94         }
95
96     def _select_for_handoff(self, windows: List[Dict[str, Any]],
97                           window_stats: List[Dict[str, Any]],
98                           idx_cfg: Dict[str, Any]) -> List[int]:
99         """Indices of the candidate windows that proceed to the AI handoff (and thus may
100         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
101         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
102         Persisted candidates are NOT affected ? they remain the full superset for
offline
103         re-scoring."""
104         return list(range(len(windows)))
105
106     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
107                     player_pool: Optional[List[str]] = None,
108                     max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
109         # override-ignore: the ABC declares the Result contract (Success|Failure)
110         # but every live segmenter still returns a bare list ? the documented
111         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
112         label = self.display_label
113         # --- Sport profile + AI provider wiring (identical across segmenters) ---
114         sport_name = self.config.get("sport", "badminton")
115         try:
116             sport_profile = sport_registry.get(sport_name)
117         except KeyError as e:
118             logger.error(str(e))
119             return []
120
121         idx_cfg = self.config.get("indexing", {})
122         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
123         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
124
125         api_key_env_var = f"{provider_name.upper()}_API_KEY"
126         api_key = os.environ.get(api_key_env_var)
127         if not api_key:
128             logger.error(
129                 f"{api_key_env_var} environment variable is not set! "
130                 f"To run the {provider_name} handoff, set your API key. "
131                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
132             )
133             return []
134         try:
135             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
136         except KeyError as e:
137             logger.error(str(e))
138             return []
139
140         video_duration = get_video_duration(video_path)
141         if video_duration <= 0:
142             logger.error("Invalid video duration.")
143             return []
144         fps, frame_width = self._probe_fps_width(video_path)
145
146         # --- Runner config + windowing thresholds ---
147         runner, runner_params = self._build_runner(idx_cfg)
148

```

```

149     velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
150     merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
151     min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
152     handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
153     ai_max_workers = idx_cfg.get("ai_max_workers", 5)
154     log_candidates = idx_cfg.get("log_yolo_candidates", True)
155     store_candidates = idx_cfg.get("store_yolo_candidates", True)
156
157     detection_params = {
158         "detector": self.name,
159         "velocity_thresh": velocity_thresh,
160         "merge_gap": merge_gap,
161         "min_action_window_duration": min_window_duration,
162         **runner_params,
163     }
164
165     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
166     ok, msg = runner.healthcheck()
167     if not ok:
168         logger.error("%s WSL environment not ready: %s", label, msg)
169         return []
170     logger.info("%s WSL env OK: %s", label, msg)
171
172     # --- Phase 2: trajectory -> candidate rally windows ---
173     # Trajectory detectors process the whole video in one pass (they carry
174     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
175     t_start = time.time()
176     out_dir = os.path.join("output", self.family)
177     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
178     if not csv_path:
179         logger.error("%s produced no trajectory; aborting.", label)
180         return []
181
182     points = runner.parse_trajectory_csv(csv_path)
183     logger.info("Parsed %d trajectory points (%d visible).",
184                 len(points), sum(1 for p in points if p.visible))
185
186     all_candidate_windows = runner.trajectory_to_action_windows(
187         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
188         velocity_thresh=velocity_thresh, merge_gap=merge_gap,
189         min_window_duration=min_window_duration, chunk_index=0,
190     )
191     logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
192                 label, time.time() - t_start, len(all_candidate_windows))
193
194     if log_candidates:
195         for cw in all_candidate_windows:
196             logger.info(f"[{label} rally]
197 {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
198 f"({cw['end'] - cw['start']:.1f}s) |
199 frames={cw['active_frame_count']} "
200 f"peak_v={cw['peak_velocity']:.3f}
201 mean_v={cw['mean_velocity']:.3f}")
202
203     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
204     # segmenter adds net_crossings + person-cue features. Computed once, reused for
205     # persistence and the handoff gate below.
206     window_stats = [
207         self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,

```

```

208         # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
209         candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
210         if store_candidates:
211             for i, cw in enumerate(all_candidate_windows):
212                 candidate_ids[i] = self.db.add_candidate_segment(
213                     video_id, detector=self.name,
214                     start_time=cw["start"], end_time=cw["end"],
215                     chunk_index=cw["chunk_index"], confidence=min(1.0,
216                     cw["peak_velocity"]),
217                     stats=window_stats[i], detection_params=detection_params,
218                 )
219         if not all_candidate_windows:
220             return []
221
222         # --- Phase 3: AI provider handoff over padded candidate clips ---
223         # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
224         # candidates above stay the full superset ? only the served play_segments are
225         # gated.
226         handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
227         idx_cfg)
228         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
229         os.makedirs(handoff_dir, exist_ok=True)
230         logger.info("Phase 3: %s validation on %d candidate windows...",
231         provider_name, len(handoff_indices))
232
233         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
234             w_start = max(0, window["start"] - handoff_padding)
235             w_end = min(video_duration, window["end"] + handoff_padding)
236             w_dur = w_end - w_start
237             w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
238             if not extract_video_chunk(video_path, w_start, w_dur, w_path,
239             reencode=True):
240                 return []
241             prompt = sport_profile.get_prompt(chunk_duration=w_dur)
242             segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
243             label=f"Handoff {wi+1}")
244             valid = []
245             for seg in segments:
246                 try:
247                     seg["start_time"] = float(seg["start_time"]) + w_start
248                     seg["end_time"] = float(seg["end_time"]) + w_start
249                     seg["candidate_id"] = candidate_ids[wi]
250                     valid.append(seg)
251                 except (ValueError, TypeError, KeyError) as e:
252                     logger.warning("Dropping segment with unparseable timestamps: %s ?
253             %s", seg, e)
254             return valid
255
256         ai_segments: List[dict] = []
257         with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
258         executor:
259             futures = [executor.submit(process_candidate_window, wi,
260             all_candidate_windows[wi])
261             for wi in handoff_indices]
262             for future in futures:
263                 ai_segments.extend(future.result())
264         try:
265             os.rmdir(handoff_dir)
266         except OSError:

```

```

265         pass
266         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
267
268         # --- Phase 4: DB population + candidate linking ---
269         segments_data = []
270         for segment in ai_segments:
271             try:
272                 start = float(segment["start_time"])
273                 end = float(segment["end_time"])
274                 dur = end - start
275                 metadata = {
276                     "shot_count": self._shot_count_from(segment, dur),
277                     "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
278                     "detector": f"{self.family}-hybrid-{model_name}",
279                 }
280                 segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
281                 if segment_id:
282                     candidate_id = segment.get("_candidate_id")
283                     if candidate_id is not None:
284                         self.db.link_candidate_to_segment(candidate_id, segment_id)
285                     segments_data.append({
286                         "id": segment_id, "start_time": start, "end_time": end,
287                         "duration": dur, "metadata": metadata,
288                     })
289             except Exception as e:
290                 logger.warning("Skipping invalid segment struct: %s", e)
291
292         return segments_data
293

```

9. assistant (2026-06-13T19:15:07.651Z)

I have now confirmed the full call path. Let me verify the key claims in the finding precisely.

The finding states:

1. `\_enrich\_candidate\_stats` calls `gate\_windows(points, [(cw['start'], cw['end'])], fps, min\_crossings=0)` separately for every window ? **CONFIRMED** at fusion_hybrid.py:77, called within the list comprehension at trajectory_hybrid.py:203-206 which iterates over `all\_candidate\_windows`.
2. `gate\_windows` internally calls `estimate\_play\_axis(points)` on the entire trajectory each time ? **CONFIRMED** at rally_gate.py:96. This sorts all visible xs and ys (via `\_spread` ? `\_sorted` at rally_gate.py:41, and `\_median` ? `\_sorted` at rally_gate.py:48). So it's $O(N \log N)$ per call.
3. `\_points\_in(points, ...)` does a full scan ? **CONFIRMED** at rally_gate.py:88, a full list comprehension over all points.

So per window: `estimate\_play\_axis` is $O(N \log N)$ over the whole trajectory, plus `\_points\_in` is $O(N)$ over the whole trajectory. Repeated $O(\text{num_windows})$ times. The finding's complexity claim $O(\text{windows} \times \text{points} \times \log \text{points})$ is accurate.

Let me double check correctness ? that recomputing axis/net yields the same value each time (so it's purely a perf smell, not a bug). `estimate\_play\_axis(points)` is a pure function of `points`, and `points` is the same whole-video trajectory passed in for every window (trajectory_hybrid.py:204 passes the same `points` to each call). So yes ? deterministic, same result each call. Correctness holds; it's purely performance.

Let me verify there isn't already caching that would refute the "repeated each time" claim ? i.e., check whether `estimate\_play\_axis` or `gate\_windows` memoizes.

10. user (2026-06-13T19:15:09.217Z)

No matches found

11. user (2026-06-13T19:15:10.176Z)

```
backend\eval\distill_local.py:56: gated = rally_gate.gate_windows(points, intervals, fps,
min_crossings=0) # for net-crossings
backend\pipeline\detectors\rally_gate.py:55: def estimate_play_axis(points) -> Tuple[str, float,
float]:
backend\pipeline\detectors\rally_gate.py:91: def gate_windows(points, windows: List[Interval],
fps: float,
backend\pipeline\detectors\rally_gate.py:96: axis, net, span = estimate_play_axis(points)
backend\pipeline\detectors\rally_gate.py:112: gate_windows(points, windows, fps,
min_crossings, deadband_frac) if g.kept]
backend\pipeline\segmenters\fusion_hybrid.py:77: gated = rally_gate.gate_windows(points,
[(cw["start"], cw["end"])], fps, min_crossings=0)
```

12. assistant (2026-06-13T19:15:21.328Z)

No caching exists. And critically, `backend/eval/distill\_local.py:56` shows the *correct* pattern that refutes any "this is intended" defense ? there, the caller passes ALL `intervals` to a SINGLE `gate\_windows(points, intervals, ...)` call. So `gate\_windows` already supports batching multiple intervals (rally_gate.py:91 takes `windows: List[Interval]` and loops internally, estimating axis once at line 96). The fusion code instead calls it once-per-window with a single-element list, which is exactly the inefficiency the finding describes, and the suggested fix (one batched call) is already proven elsewhere in the codebase.

Let me confirm there's no per-window axis variation intent by checking that `points` passed at the call site is genuinely the whole-video trajectory and not a per-window slice (which would make re-estimation necessary, refuting the finding).

The call site at trajectory_hybrid.py:204 is `self.\_enrich\_candidate\_stats(cw, points, fps, frame\_width, video\_path, idx\_cfg)`, where `points = runner.parse\_trajectory\_csv(csv\_path)` (line 182) ? the whole-video trajectory, identical across all iterations. Confirmed.

The finding holds. Let me assess severity. This is a genuine, real performance smell:

- It does NOT affect correctness (axis/net is deterministic over the same `points`).
- It is only triggered on the opt-in `fusion\_hybrid` segmenter (default-OFF), and only matters for long matches with many candidate windows and dense trajectories.
- The waste is $O(\text{windows} \times N \log N)$ where it could be $O(N \log N + \text{windows} \times \text{window_points})$.
- A clean batched API (`gate\_windows(points, intervals, ...)`) already exists and is used correctly in `distill\_local.py`, so the fix is low-risk and well-precedented.

The finding's own severity is "low" and explicitly frames it as a performance smell, not a bug. That self-assessment is honest and accurate. It is real but minor.

13. user (2026-06-13T19:15:36.373Z)

Structured output provided successfully

14. assistant (2026-06-13T19:15:44.145Z)

Verified the finding against the actual code. It is REAL but low severity.

Key evidence:

- `backend/pipeline/segmenters/fusion\_hybrid.py:77` calls `gate\_windows(points, [(cw["start"], cw["end"])], fps, min\_crossings=0)` with a single-element interval list.
- It runs once per window via the comprehension at `backend/pipeline/segmenters/trajectory\_hybrid.py:203-206`, with the same whole-video `points` (`parse\_trajectory\_csv` result, line 182) each time.
- Inside `gate\_windows` (`backend/pipeline/detectors/rally\_gate.py:96`), `estimate\_play\_axis(points)` sorts all visible xs/ys ($O(N \log N)$) over the entire trajectory, and `\_points\_in` (`rally\_gate.py:88`) does a full $O(N)$ scan ? both repeated per window.

Correctness is unaffected (deterministic pure function on identical input), so it is purely a

perf smell, exactly as the finding claims. The suggested fix is already used correctly elsewhere: ``backend/eval/distill_local.py:56`` passes the full interval list to a single ``gate_windows`` call, and the function signature (``rally_gate.py:91``) already supports batching with a single axis estimate. Low severity is the honest rating given the segmenter is default-OFF and runs owner-side where the AI handoff dominates cost.